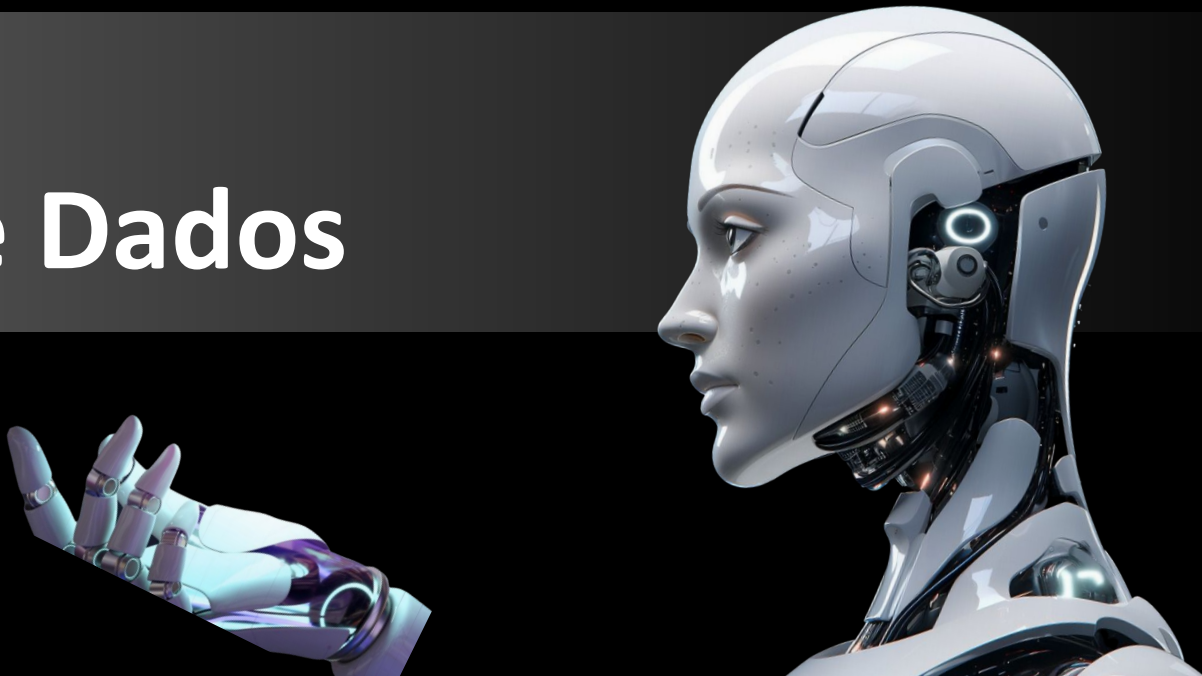




PROJETO EDUCAÇÃO
DO FUTURO

Banco de Dados



Sumário

- Grupos de comandos
- Quais os tipos de categoria que existem no SQL
- Entidades
- Restrições de Integridade
- Atividades



Grupos de comandos do Banco de Dados

Na aula passada vimos os principais comandos de consulta do SQL como: **SELECT, ORDER BY, WHERE** dentre outros.

Mas uma coisa que é importante saber é que esses comandos se encaixam na modalidade de comando conhecidos como: **DQL**

DQL - Data Query Language

O DQL é uma vertente do SQL que como o nome já diz é feito para manipular os nossos dados (DQL - Linguagem de Consulta de Dados).

Em suma, tudo que usamos para fazer consultas dentro do banco podem ser considerados como comandos DQL.

- **SELECT**
- **WHERE**
- **FROM**

Quais os tipos de categoria que existem no SQL?

Após isso você pode está se perguntando quais os tipos de categorias que existem no SQL? Bom os que existem são:

- **DQL:** linguagem de consulta de dados. Para fazermos consultas e demais filtragens.
- **DDL:** Linguagem de definição de dados (Data Definition Language), pois serve para criarmos nossas tabelas, bases de dados, procedures e demais objetos do banco de dados.
- **DML:** Linguagem de manipulação de dados (Data Manipulation Language) que serve para adicionar, inserir, excluir e atualizar nossos dados.

DDL - CREATE DATABASE

DDL como falado anteriormente serve para criarmos elementos dentro do nosso banco de dados. Esses elementos podem ser: tabelas, base de dados e etc.

Para criar nossa base de dados (schema) para guardarmos nossas tabela teremos que usar o seguinte comando.

Um exemplo de comando DDL seria:

```
CREATE DATABASE nomeBaseDados
```

DDL - CREATE

O comando **CREATE** serve para criarmos elementos dentro do banco de dados. Já vimos anteriormente esse comando para criar base de dados, mas iremos usá-lo principalmente para criar tabelas.

Na hora de criarmos tabelas podemos também criar colunas e especificar qual o tipo de dado cada uma delas irá receber.

Necessitamos lembrar que sempre que criamos uma tabela queremos representar algo do mundo real para o nosso sistema.

Vamos supor que eu quero modelar uma escola usando o SQL para isso eu teria que pensar em tudo que existe dentro de uma escola: professor, aluno, coordenador dentre outros.

Entidades

Sempre que você está fazendo essa transformação de algo real para o seu banco de dados você vai se deparar com o conceito de entidades.

Entidades nada mais são do que um grupo de informações que representam algo do meu sistema e eu irei representar no formato de tabela.

Por exemplo: Seguindo a nossa ideia de modelar uma escola com o SQL chegaremos em um determinado ponto onde eu preciso representar um aluno para o meu sistema.

Para fazer isso, precisamos pensar o seguinte: O que um aluno necessita ter para ser considerado um aluno? A resposta sempre dependerá do tipo de sistema que você está tendo que modelar, mas vamos considerar o seguinte:

Entidades

Nosso aluno precisa ter as seguintes características:

- Nome
- matricula
- Curso
- data_inicio
- data_conclusão

Como eu faria para representar isso no SQL? Para fazer isso eu necessitaria me utilizar do seguinte comando **CREATE** como vimos anteriormente, porém ele virá acompanhado do comando **TABLE**.

DDL - CREATE TABLE

O comando **CREATE TABLE** permite criarmos nossas tabelas e definirmos os nossos campos dentro dela.

Porém nosso trabalho não está completo, pois ainda precisamos definir o que cada campo irá receber.

Ao contrário de ferramentas como o Excel o banco de dados não aceita que tabelas tenham valores misturados

Por exemplo: Imagine uma coluna de salário do excel tendo um nome escrito dentro dele ou qualquer outro tipo de dado que não seja numérica.

O SQL nos obriga a dizer para ele o que cada campo aceita e se tentarmos forçar-lo a aceitar alguma outra coisa ele irá negar e retornar um erro

DDL - CREATE TABLE

Para criarmos nossa tabela precisamos seguir o comando abaixo, porém como falado anteriormente ainda temos que tipificar o nosso dado para o SQL saber o que cada campo irá aceitar.

```
CREATE TABLE aluno(  
    Nome  
    matrícula  
    Curso  
    idade  
    sexo  
    nota  
)
```

Quantos tipos de dados existem dentro do SQL?

Isso depende da versão do MYSQL que você está utilizando, mas neste momento existem no total 40 tipos de dados dos mais diversos possíveis.

O SQL possui uma variedade de dados extensa que podemos utilizar para montar nossas tabelas.

Alguns desses dados já temos conhecimento como o INT e FLOAT, porém aqui existe uma diferença, pois o SQL faz uma "família/conjunto" daqueles tipos de dados.

Classificação do tipo de dado Numérico

tipo	tamanho	Range (Assinado)	Range (não assinado)	uso
TINYINT	1 byte	(-128.127)	(0255)	valores inteiros pequenos
SMALLINT	2 bytes	(768,32 -32 767)	(535 0,65)	valor inteiro
MEDIUMINT	3 bytes	(-8388 608,8 388 607)	(0,16 777.215)	valor inteiro
INT ou inteiro	4 bytes	(-2 147 483 648,2 147 483 647)	(0,4 294 967 295)	valor inteiro
BIGINT	8 bytes	(-9.233.372.036.854.775 808,9 223.372.036.854.775 807)	(0,18 446.744.073.709.551 615)	valor máximo inteiro
FLOAT	4 bytes	(-3,402 823 466 E + 38,1.175 494 351 E-38), 0, (1.175 494 351 E-38,3.402 823 466 351 E + 38)	0, (1.175 494 351 E-38,3.402 823 466 E + 38)	valores de ponto flutuante de precisão simples
DUPLO	8 bytes	(1,797 693 134 862 315 7 E + 308,2.225 073 858 507 201 4 E-308), 0, (2,225 073 858 507 201 4 E-308,1.797 693 134 862 315 7 E + 308)	0, (2,225 073 858 507 201 4 E-308,1.797 693 134 862 315 7 E + 308)	valores de ponto flutuante de precisão dupla
DECIMAL	De DECIMAL (H, D), se M > D, M + 2 é outra forma D + 2	Depende dos valores de M e D	Depende dos valores de M e D	valor decimal

Classificação do tipo de dado Strings

tipo	tamanho	uso
CHAR	0-255 bytes	cadeia de comprimento fixo
VARCHAR	0-65535 bytes	cadeias de comprimento variável
TINYBLOB	0-255 bytes	Não mais do que 255 caracteres na cadeia binária
TINYTEXT	0-255 bytes	cadeias de texto curtas
BLOB	0-65535 bytes	dados longos de texto em formato binário
TEXT	0-65535 bytes	dados de texto longo
MEDIUMBLOB	0-16777215 bytes	forma binária de dados de texto de comprimento médio
MEDIUMTEXT	0-16777215 bytes	dados de texto de comprimento médio
LONGBLOB	0-4294967295 bytes	Grandes dados de texto em formato binário
LONGTEXT	0-4294967295 bytes	Grande dados de texto

Classificação do tipo de dado Data

tipo	tamanho (Byte)	escopo	formato	uso
DATA	3	1000/01/01 / 9999-12-31	AAAA-MM-DD	valores de data
TEMPO	3	'-838: 59: 59' / '838: 59: 59'	HH: MM: SS	valor do tempo ou a duração
ANO	1	1901/2155	AAAA	Valor ano
DATETIME	8	1000-01-0100: 00: 00 / 9999-12-31 23:59:59	AAAA-MM-DD HH: MM: SS	Mistura de data e hora valores
TIMESTAMP	4	Algum 00/2037 Ano: 1970-01-01 00:00	AAAAMMDD HHMMSS	Misturando data e valor de tempo, um timestamp

SQL - CREATE TABLE

Podemos colocar qual o tipo de dado cada coluna irá receber, mas para isso precisamos escrever o nome da coluna e em sequência iremos explicitar qual tipo de dado aquela coluna recebe

```
CREATE TABLE NomeTabela(  
    Nome VARCHAR(50)  
    matricula INT  
    Curso VARCHAR(50)  
    Sexo ENUM('M', 'F'),  
    idade INT,  
    nota FLOAT  
)
```


Restrições de Integridade

Restrições de integridade são maneiras que temos para garantir a segurança, exatidão e consistência dos dados da nossa tabela.

Existem diversas restrições de integridade, mas as mais importantes são

- Restrição de chave:
- Restrição de vazio
- Restrição de domínio

Chave Primária (Restrição de chave)

A chave primária é um dos conceitos mais importantes que precisamos aprender no SQL, pois é ele que irá preencher nossa restrição de chave, pois ela permite colocarmos campos que terão valores que só podem ser únicos, ou seja, só aparecem uma vez

Para ficar mais claro imagine o seu cpf, agora pense no seguinte: “existe mais alguém em todo o Brasil que possa ter o meu cpf?” Não! E isso é a chave primária! Ela é algo que pertence só a você e a mais ninguém.

Isso serve como um localizador universal, pois ao invés de pesquisarmos por algo genérico como nome estamos pesquisando por algo muito mais preciso.

Chave Primária

Existem algumas regras para a criação da nossa chave primária e elas são:

- Uma coluna com chave primária não pode se repetir tendo em vista que ela é o “CPF” da nossa tabela.
- Tem que ser única.
- Só pode existir uma chave por tabela.

Chave Primária

```
CREATE TABLE NomeTabela(  
    Nome VARCHAR(50) NOT NULL,  
    matricula INT PRIMARY KEY,  
    Curso VARCHAR(50),  
    Sexo ENUM('M', 'F'),  
    Idade INT,  
    Nota Float  
)
```

Restrição de Vazio

Essa restrição permite que possamos dizer se um campo é obrigatório e que deve ser preenchida pelo usuário

Sua sintaxe é a seguinte:

```
CREATE TABLE NomeTabela(  
  Nome VARCHAR(50) NOT NULL,  
  matricula INT PRIMARY KEY,  
  Curso VARCHAR(50),  
  Sexo ENUM('M', 'F'),  
  idade INT,  
  Nota Float  
)
```

Restrição de domínio

A restrição de domínio é referente ao tipo de dados que aquela coluna pode suportar.

Se eu falo que a coluna idade é do tipo INT ela só pode receber dados do domínio numérico inteiro.

Nome **VARCHAR(50)** – Só pode texto com até 50 caracteres

matricula **INT PRIMARY KEY**, – Só permite número inteiro

Curso **VARCHAR(50)**, – Só pode texto com até 50 caracteres

Sexo **ENUM('m', 'f')**, – Só aceita valores predefinidos

idade **INT** – Só permite dados inteiro

Nota **FLOAT** – Só permite números reais

AUTO_INCREMENT

O comando AUTO_INCREMENT é extremamente útil quando estamos nos utilizando de chave primária e estrangeira para trabalharmos, pois ele permite que a cada nova inserção no nosso banco de dados o valor da coluna seja incrementado automaticamente'

```
CREATE TABLE Aluno(  
  Nome VARCHAR(50) NOT NULL,  
  matricula INT PRIMARY KEY AUTO_INCREMENT,  
  Curso VARCHAR(50),  
  sexo ENUM('M', 'F'),  
  idade INT,  
  Nota FLOAT  
);
```

Como inserir dados dentro da nossa base

Conseguimos criar a nossa base de dados com sucesso, porém como faremos para inserir dados dentro da nossa tabela que acabamos de criar?

Bom, para fazermos isso usaremos o comando conhecido como **INSERT INTO** ele irá nos permitir inserir dados dentro de determinadas colunas da nossa tabela selecionada.

DDL - INSERT INTO

O comando **INSERT INTO** permite que possamos inserir dados dentro da nossa tabela.

Seu comando é:

```
INSERT INTO  
    NomeTabela (coluna1,coluna2,coluna3)  
VALUES (  
    ValorColuna1, ValorColuna2, ValorColuna3  
)
```

DDL - ALTER TABLE

O comando **ALTER TABLE** permite a nós fazermos alterações dentro da nossa tabela das maneiras mais diversas possíveis

O comando base para isso é:

```
ALTER TABLE
```

```
  Tabela
```

```
ADD COLUMN
```

```
  NomeColuna TipoDeDado
```

DDL - ALTER TABLE DROP COLUMN

O DROP COLUMN permite a nós droparmos(removermos) uma coluna no SQL para fazer basta realizar o seguinte:

```
ALTER TABLE
```

```
    Tabela
```

```
DROP COLUMN
```

```
    NomeColuna
```

DDL - RENAME TABLE

O comando RENAME TABLE serve para conseguirmos renomear uma das nossas tabelas

```
RENAME TABLE TabelaAntiga TO NovoNome
```

DDL - MODIFY

O comando MODIFY permite que possamos modificar as colunas de suas tabelas, podendo assim mudar os tipos de dados que aquela coluna aceita

```
ALTER TABLE Tabela MODIFY NomeColuna INT
```

UPDATE

O comando UPDATE consegue fazer atualizações no nosso banco de dados para podermos realizar alterações dentro dele quando necessário.

Para realizarmos nossas alterações podemos usar o seguinte comando:

```
UPDATE
    NomeTabela
SET
    AlteracaoColuna = Alteracao
WHERE
    matricula = 2;
```

Comando DROP

O comando DROP com certeza é o comando mais perigoso, pois se não usado com extrema cautela pode simplesmente destruir nossa tabela ou banco de dados já que a única ação dele dentro do banco de dados é de remover os elementos contidos dentro dele.

```
DROP TABLE NomeTabela
```

DELETE

O comando DELETE deve ser usado com cuidado, pois quando utilizado sem a cláusula WHERE poderá destruir toda sua base de dados.

O que ele faz seu próprio nome já indica ele deleta uma base de dados

DELETE FROM NomeDaTabela **WHERE** Coluna = Condição

DDL - TRUNCATE

O comando TRUNCATE permite deletar todos os registros de uma tabela sem apagar a tabela em si

```
TRUNCATE TABLE NomeDaTabela
```

EXERCÍCIO

1. Adicione 10 alunos dentro da sua tabela 'ALUNO'
1. Altere o curso do aluno com a matrícula número 5 para DS(Data Science)
1. Delete os dados do aluno de matrícula 10
1. Ordene as Notas dos alunos da maior até a menor

EXERCITANDO

Agora com isso em mente crie uma tabela chamada FUNCIONÁRIOS que tenha as seguintes colunas:

OBS: quantidade de caracteres ficam a escolha de vocês

Código	Nome	SETOR	Salário
INT PK AU	VARCHAR(*)	VARCHAR(*)	DECIMAL(10,2)

EXERCÍCIO

- Agora sabendo dos comandos de inserção dentro do banco de dados resolva as seguintes atividades

1. Resolva os seguintes itens:

A. Insira dentro da sua tabela dez registros nas colunas de:
Nome ,Setor ,Salário

Considere os seguintes setores:

- TI
- Manutenção
- Financeiro
- RH

EXERCÍCIO

1. Resolva os seguintes itens:

- B Pegue o maior salário existente entre todos os empregados
- C Pegue o menor salário existente entre todos os empregados
- D Altere o salário do empregado com o código 3 para 5000

EXERCÍCIO

2. Resolva os seguintes itens:
 - A. Média geral do salário de funcionários
 - A. Ordene os empregados por ordem alfabética
 - A. Conte quantos empregados existem por SETOR